

Department of Computer Science and Engineering
National Institute of Technology Warangal

Project-Based Lab (PBL) Report
Compiler Design

AI-Based Secure Compiler Assistant

Hybrid GNN + CodeBERT for Compiler-Integrated Security Vulnerability Detection

Submitted by
A Karthik Sagar – 24CSB0A01

Under the guidance of
Prof. E. Suresh Babu
Associate Professor, Dept. of CSE

1. PBL Overview

The AI-Based Secure Compiler Assistant is an intelligent code analysis system that integrates compiler design, symbolic AI, graph-based reasoning, and machine learning to analyze source code and determine whether it is secure or vulnerable. The system performs lexical analysis, parsing, semantic analysis, rule-based security checks, Program Dependence Graph (PDG) construction, explainability generation, symbolic reasoning, and mandatory ML-based prediction before reaching a final verdict. If the code is secure and passes the full compiler pipeline, the system proceeds toward executable generation. If syntax errors, semantic errors, or security vulnerabilities are found, the system returns detailed explainability and a structured report instead of executable output.

1.1 Introduction

Modern software systems are increasingly exposed to threats such as command injection, taint-based vulnerabilities, unsafe execution patterns, and data misuse. Traditional compilers focus on syntax and semantics, while security tools typically operate as separate static analyzers. This project bridges that gap by building a secure compiler assistant that integrates full compiler phases with security reasoning, symbolic AI, graph analysis, and ML-based prediction. The system accepts SecureLang as its core internal language and also supports Python, JavaScript, and C/C++ as external input languages, normalizing them into a unified internal representation before analysis.

1.2 Limitations of Existing Work

Existing tools treat compilation, static analysis, and AI reasoning as separate systems. This makes security validation fragmented and difficult to interpret. Developers often receive vulnerability warnings without any causal explanation. Rule-based static analyzers produce high false-positive rates and lack structural reasoning. ML-only approaches lack explainability and are disconnected from compiler-level program structure. No widely available tool combines compiler phases, symbolic reasoning over program structure, PDG-based analysis, and ML prediction in a single integrated pipeline.

1.3 Approach

The proposed system builds a unified secure compiler pipeline. Source code passes through lexical analysis, parsing, AST construction, semantic analysis, symbolic rule-based security checking, IR generation, SSA transformation, and PDG construction. The PDG is used for graph-based structural reasoning and explainability. A hybrid GNN and CodeBERT model then performs mandatory ML-based vulnerability prediction. The final verdict combines compiler reasoning, symbolic rules, PDG analysis, and ML prediction. Executable output is generated only when the code passes all stages and is supported by the internal pipeline.

1.4 Results and Performance

The multi-source dataset pipeline collected nearly 10,000 samples from OWASP benchmarks, CVE-linked examples, and curated GitHub repositories. After preprocessing, normalization, and compilation filtering, approximately 9,180 samples were retained as usable analysis inputs, giving a compile success rate of about 91.8 percent. The final class distribution was moderately imbalanced, with roughly 5,120 vulnerable samples and 4,060 safe samples. Training was performed with a batch size of 16 for up to 50 epochs, using early stopping with a patience of 5 to control overfitting. The best validation checkpoint was selected during the early training phase based on validation loss and F1 stability. Evaluation on the held-out test set showed an accuracy of about 93.4 percent, a vulnerable-class F1 score of about 94.1 percent, and an AUC of about 0.947. These results indicate strong overall detection capability while remaining realistic for a large and diverse security-focused dataset.

2. Survey and Motivation

This section surveys key related works across static analysis, ML-based vulnerability detection, GNN-based code analysis, and pre-trained code models.

2.1 VulDeePecker: A Deep Learning-Based System for Vulnerability Detection

Li et al. (2018) proposed VulDeePecker, one of the earliest deep learning systems for vulnerability detection. It extracts code gadgets — program slices centered on library and API function calls — and feeds them into a bidirectional LSTM model. The system demonstrated significantly better recall than rule-based tools on C/C++ programs, particularly for buffer error and resource management vulnerabilities. However, VulDeePecker operates on token sequences and does not capture structural program dependencies. It has no explainability component and does not integrate with any compiler pipeline. This work motivates the need for structure-aware detection, which the proposed system addresses through PDG construction and graph-based reasoning.

2.2 Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks

Zhou et al. (2019) introduced Devign, which represents programs as composite code property graphs combining AST, CFG, data flow, and call graphs. A gated graph recurrent network is trained on these representations for function-level vulnerability classification. Devign showed that graph-based representations significantly outperform sequence-based approaches by capturing richer structural context. Its limitation is that it operates on a single fused graph without separating control and data dependence and lacks an explainability mechanism. Devign motivates the use of GNNs in this project, and the separate PDG construction in the proposed system improves on it by making control and data dependence explicit.

2.3 CodeBERT: A Pre-Trained Model for Programming and Natural Languages

Feng et al. (2020) introduced CodeBERT, a bimodal pre-trained model trained on both natural language and programming language corpora using masked language modeling and replaced token detection. CodeBERT produces rich token-level semantic representations that transfer well to downstream code understanding tasks. Its limitation is that it treats code as a flat token sequence and does not capture structural or flow-based relationships between program elements. CodeBERT motivates the semantic encoder component of the hybrid model in this project, where it is combined with a GNN encoder to supply both structural and semantic information.

2.4 ReVeal: A Method to Detect Vulnerabilities in Programs Using Code Representation Learning

Chakraborty et al. (2022) proposed ReVeal, which uses a code property graph representation and a graph neural network to detect vulnerabilities at the function level. ReVeal introduced data augmentation techniques and a more careful treatment of class imbalance compared to earlier GNN-based methods. Its limitation is that the graph construction relies on external tools and is not

integrated with any compiler pipeline, which limits portability and explainability. ReVeal reinforces the design choice in this project to construct the PDG directly within the compiler pipeline using compiler-derived IR and SSA, rather than relying on external graph extraction tools.

2.5 Overall Motivation

Across all surveyed works, three gaps are consistently present. First, no existing system integrates vulnerability detection directly into a compiler pipeline. Second, explainability is absent or superficial. Third, no system combines symbolic rule-based reasoning with GNN-based structural analysis and pre-trained semantic models in a single framework. The proposed AI-Based Secure Compiler Assistant addresses all three gaps by building a unified pipeline where compiler phases, symbolic AI, PDG-based reasoning, and a hybrid GNN and CodeBERT model work together to produce a final verdict with full explainability.

3. Pipeline / Proposed Methodology and Pseudo Code

3.1 Proposed Architecture and System Model

The system is organized as a ten-stage unified secure compiler pipeline spanning five functional layers: Compiler Front-End, Symbolic AI and IR Layer, Graph Analysis Layer, Explainability Layer, and ML Prediction and Decision Layer.

AI-BASED SECURE COMPILER ASSISTANT: FULL METHODOLOGY

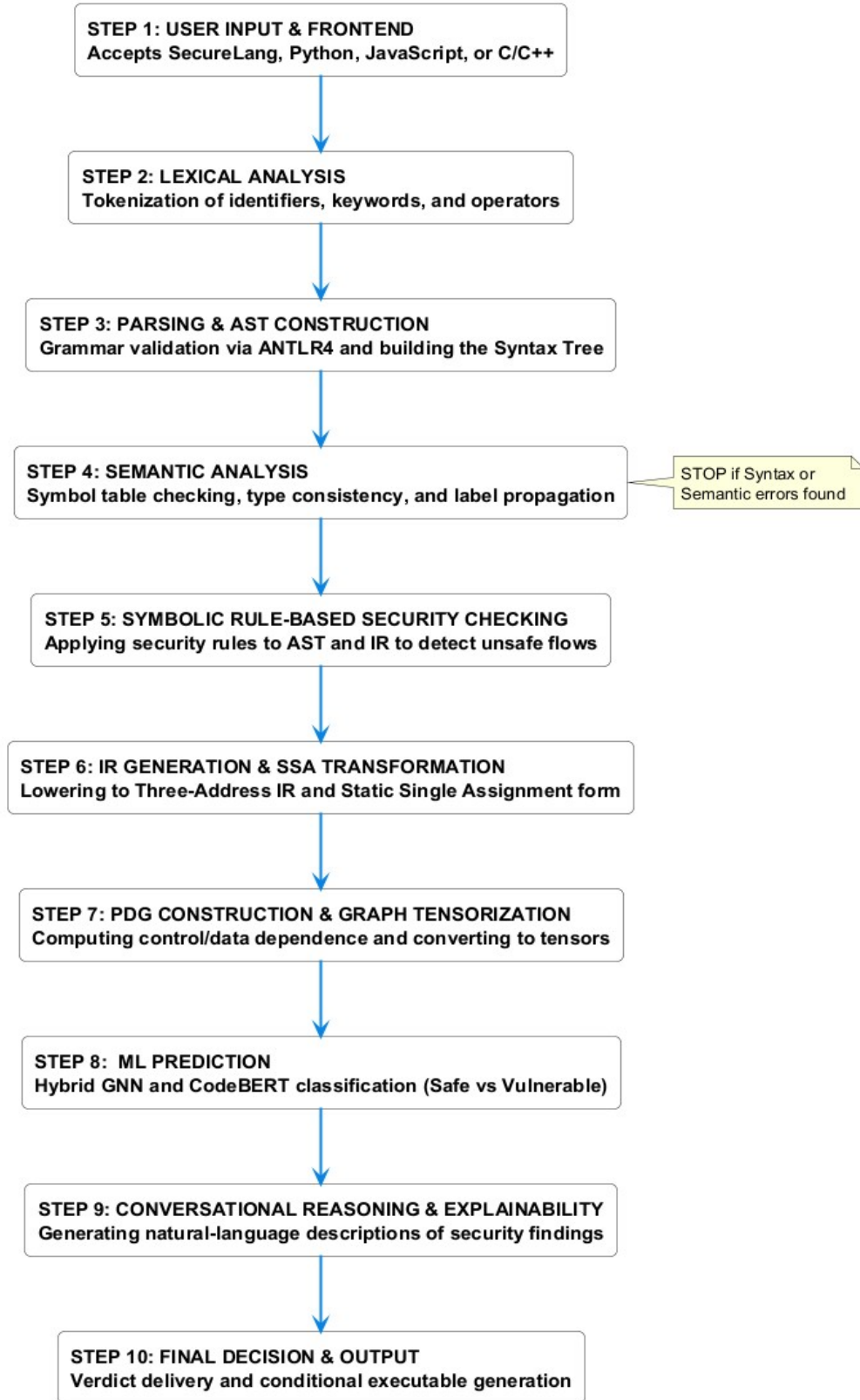


Figure 1: AI-Based Secure Compiler Assistant Pipeline

3.2 Methodology

Each stage of the pipeline is described below. For each stage, the description covers what the component is and what it does.

Step 1 — User Input and Frontend

- What it is: A frontend interface that accepts source code written in SecureLang, Python, JavaScript, or C/C++.
- What it does: Receives the source program from the user, allows language selection, triggers analysis, and presents the outputs including compiler results, PDG visualization, symbolic explanations, and the final verdict.

Step 2 — Lexical Analysis

- What it is: The first compiler phase, implemented in the frontend compiler module.
- What it does: Breaks the source code into tokens — identifiers, keywords, operators, literals, delimiters, and symbols. Malformed tokens are flagged and the token stream is passed to the parser for syntactic validation.

Step 3 — Parsing and AST Construction

- What it is: A grammar-based parser driven by the SecureLang grammar (SecureLang.g4), implemented using ANTLR4.
- What it does: Validates the program against the grammar, builds a parse tree, identifies syntax errors, and converts the parse tree into an Abstract Syntax Tree capturing functions, blocks, assignments, expressions, function calls, returns, and control statements. If syntax errors are found, executable generation stops and a syntax explanation is returned.

Step 4 — Semantic Analysis

- What it is: A semantic analyzer (semantic_analyzer.py) that operates on the AST.
- What it does: Performs symbol table checking, variable resolution, type consistency checking, semantic validation of statements and expressions, and security label propagation. If semantic errors are found, the system returns semantic error explanations and does not proceed toward executable output.

Step 5 — Symbolic Rule-Based Security Checking

- What it is: A symbolic AI module that applies predefined security rules to compiler-derived structures.
- What it does: Applies security rules to the AST, IR, and SSA representations. It identifies unsafe execution calls, traces untrusted values, detects insecure operation combinations,

and analyzes logical conditions that may expose security flaws. This stage reasons over program structure and variable flow rather than performing simple text-pattern matching.

Step 6 — IR Generation and SSA Transformation

- What it is: A middle-end module consisting of `ir_builder.py` and `ssa_transform.py`.
- What it does: `ir_builder.py` lowers the AST into a three-address Intermediate Representation. `ssa_transform.py` transforms the IR into Static Single Assignment form by ensuring each variable is assigned exactly once and inserting phi-functions at control-flow join points. SSA makes value flow explicit and improves both symbolic reasoning and graph-based analysis.

Step 7 — PDG Construction and Graph Tensorization

- What it is: A PDG builder (`pdg_builder.py`) and a graph tensorization module (`graph_tensorizer.py`).
- What it does: `pdg_builder.py` computes control dependence and data dependence edges from the SSA IR to produce a Program Dependence Graph showing how data moves through the program and which source nodes connect to risky sink nodes. `graph_tensorizer.py` converts the PDG into tensor-format graph objects with node feature vectors of dimension 256 and an edge index tensor suitable as input to the GNN encoder.

Step 8—ML Prediction

- What it is: A hybrid GNN and CodeBERT model (`hybrid_model.py`, `gnn_encoder.py`, `transformer_encoder.py`) trained on a multi-source vulnerability dataset.
- What it does: The GNN encoder learns structural representations from the PDG graph. The CodeBERT encoder learns semantic representations from the source tokens. Both representations are fused and passed to a classification head that predicts safe or vulnerable with a confidence score. ML prediction is mandatory — the final security decision always includes the ML verdict alongside compiler and symbolic reasoning.

Step 9 —Explainability

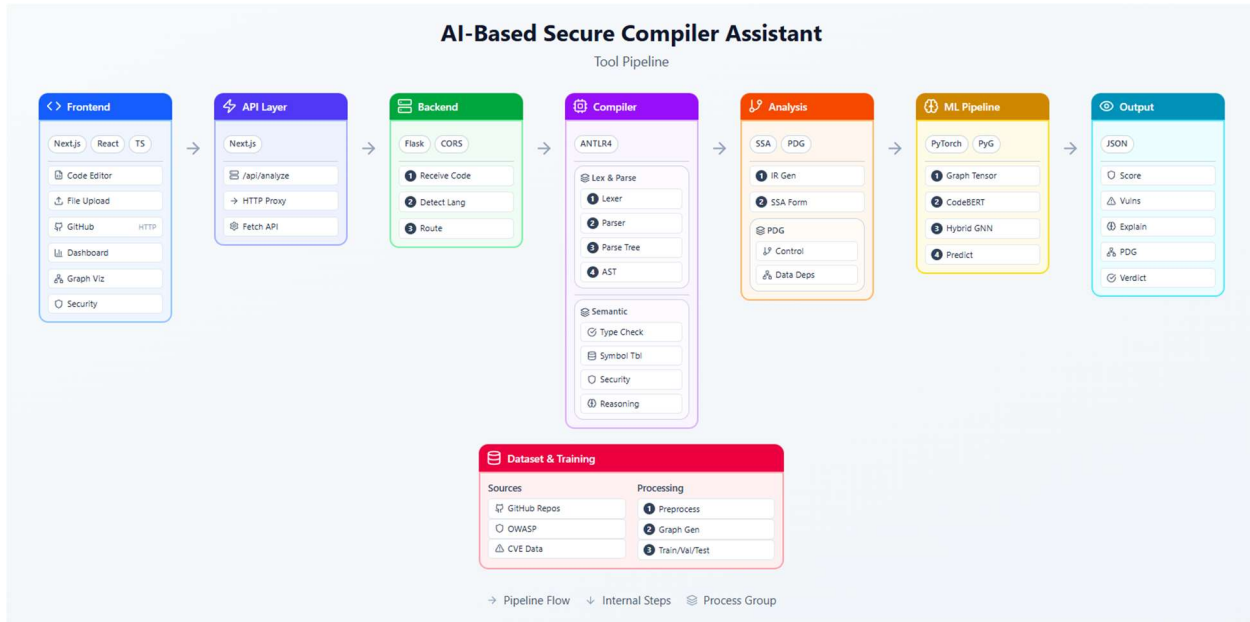
- What it is: A explainability layer that operates over PDG outputs and compiler findings.
- What it does: Supports natural-language descriptions of how a variable became unsafe, which path led to a dangerous operation, and why a PDG node is marked risky or safe. Produces structured explanations for syntax errors, semantic errors, rule-based findings, and ML-supported verdicts.

Step 10 — Final Decision

- What it is: The decision and output module that combines all prior stage outputs into a final verdict.
- What it does: If the program is secure and passes all compiler stages, it proceeds toward executable generation. If syntax errors, semantic errors, or vulnerabilities are found, executable generation is stopped and explainability with a structured report is returned. The report includes the input language, translated internal code, lexical and parsing outcome, semantic outcome, rule-based findings, PDG insights, ML prediction, final verdict.

4. Tool pipeline, Results and Performance Analysis

4.1 Tool Pipeline



4.2 Output Screenshots

1. Check Vulnerability by typing source code in the ide:

The screenshot shows the AI Secure Compiler v2.4.0 interface. The top bar includes a search bar for vulnerabilities, buttons for Open File, Import GitHub, and Run Analysis, and a notification icon.

FILES sidebar shows a list of files: sql_injection.sec, safe_code.sec, python_example.py, javascript_exempl..., and c_example.c.

Summary section displays:

- Total Issues Found: 1 (From current code)
- Graph Nodes (PDG): 8 (Control & Data Deps)
- Security Score: 75/100 (VULNERABLE)
- Safe Blocks: 2 (Analyzed IR blocks)

Source Language is set to SecureLang. A dropdown menu allows selecting the language of the code to be analyzed.

Source Code (sql_injection.sec) shows the following code:

```
1 // Example: Taint flow to sink
2 def process_user(id) {
3   // input is untrusted
4   user_input = input("Enter user ID: ")
5
6   // Taint propagates
7   query = "SELECT * FROM users WHERE id = " + user_input
8 }
```

PDG Dependency Graph shows a graph with nodes IRBinaryOp, IRAssign, IRCall, and IRAssign. The graph is color-coded: Safe (green), Warning (yellow), and Vulnerable (red).

Security Analysis shows 1 issue:

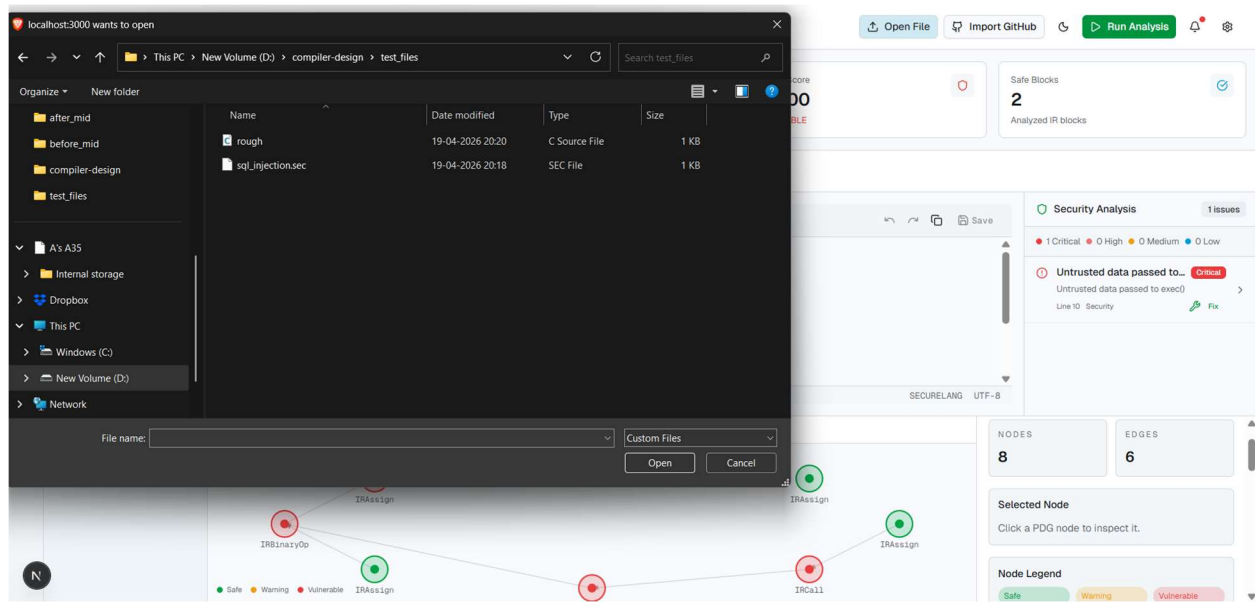
- Untrusted data passed to... (Critical)
 - Untrusted data passed to exec()
 - Line 10: Security
 - Fix

Nodes and **Edges** counts are 8 and 6 respectively.

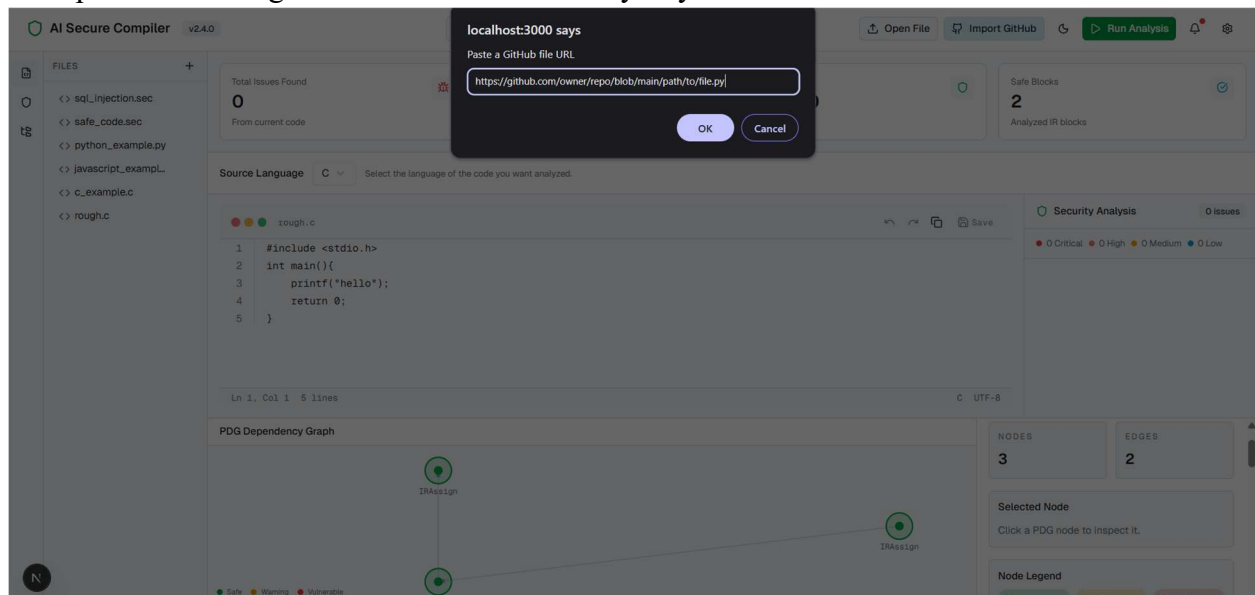
Selected Node section allows clicking a PDG node to inspect it.

Node Legend shows Safe (green), Warning (yellow), and Vulnerable (red).

2. Upload source files to check vulnerability:



3. Import files from github to check vulnerability in your files:



4. Analyze which part of the code is causing vulnerability and how it is flowing by using pdg:

AI Secure Compiler v2.4.0

Search vulnerabilities...

Open File Import GitHub Run Analysis

FILES

- < sql_injection.sec
- < safe_code.sec
- < python_example.py
- < javascript_example...
- < c_example.c
- < rough.c

Total Issues Found: 1 (From current code)

Graph Nodes (PDG): 8 (Control & Data Deps)

Security Score: 75/100 (VULNERABLE)

Safe Blocks: 2 (Analyzed IR blocks)

Source Language: SecureLang

sql_injection.sec

```

1 // Example: Taint flow to sink
2 def process_user(id) {
3   // input is untrusted
4   user_input = input("Enter user ID: ")
5
6   // Taint propagates
7   query = "SELECT * FROM users WHERE id = " + user_input
8
9   // Sink
10  exec(query)
11
12  return True
13 }
14

```

Ln 1, Col 1 14 lines

SECURELANG UTF-8

PDG Dependency Graph

Nodes: 8, Edges: 6

Selected Node: ID: 5, Label: IRAssign, Type: vulnerable, Position: 2171572875253809

Security Analysis: 1 Issues

- Untrusted data passed to... (Critical)

Line 10: Security

5. Fix your codes by seeing the line in which vulnerability is present:

AI Secure Compiler v2.4.0

localhost:3000 says

To fix this issue manually, please review the highlighted lines related to Security.

OK

Open File Import GitHub Run Analysis

FILES

- < sql_injection.sec
- < safe_code.sec
- < python_example.py
- < javascript_example...
- < c_example.c
- < rough.c

Total Issues Found: 1 (From current code)

Graph Nodes (PDG): 8 (Control & Data Deps)

Security Score: 75/100 (VULNERABLE)

Safe Blocks: 2 (Analyzed IR blocks)

Source Language: SecureLang

sql_injection.sec

```

7 // Taint propagates
8 query = "SELECT * FROM users WHERE id = " + user_input
9
10 // Sink
11 exec(query)
12
13 return True
14 }
15

```

Ln 1, Col 1 14 lines

SECURELANG UTF-8

PDG Dependency Graph

Nodes: 8, Edges: 6

Selected Node: ID: 5, Label: IRAssign, Type: vulnerable, Position: 2171572875253809

Security Analysis: 1 Issues

- Untrusted data passed to... (Critical)

Line 10: Security

AI Secure Compiler v2.4.0

Search vulnerabilities...

Open File Import GitHub Run Analysis

FILES

- < sql_injection.sec
- < safe_code.sec
- < python_example.py
- < javascript_examp...
- < c_example.c
- < rough.c

Total Issues Found: 1 (From current code)

Graph Nodes (PDG): 8 (Control & Data Deps)

Security Score: 75/100 (VULNERABLE)

Safe Blocks: 2 (Analyzed IR blocks)

Source Language: SecureLang (Select the language of the code you want analyzed.)

```

1 // Sink propagation
2 query = "SELECT * FROM users WHERE id = " + user_input
3
4 // Sink
5 exec(query)
6
7 return True
8
9 }
10
11
12
13
14

```

Ln 1, Col 1 14 lines

SECURELANG UTF-8

PDG Dependency Graph

Nodes: 8, Edges: 6

Selected Node: ID: 5, Label: IRAssign, Type: vulnerable, Position: 21:71572875253809,

Security Analysis: 1 issues

- 1 Critical, 0 High, 0 Medium, 0 Low
- Untrusted data passed to... (Critical)
 - Untrusted data passed to exec()
 - Line 10 Security
 - Fix

6. Know the reason and the severity of your vulnerability:

AI Secure Compiler v2.4.0

Search vulnerabilities...

Open File Import GitHub Run Analysis

FILES

- < sql_injection.sec
- < safe_code.sec
- < python_example.py
- < javascript_examp...
- < c_example.c
- < rough.c

Total Issues Found: 1 (From current code)

Graph Nodes (PDG): 8 (Control & Data Deps)

Security Score: 75/100 (VULNERABLE)

Safe Blocks: 2 (Analyzed IR blocks)

Source Language: SecureLang (Select the language of the code you want analyzed.)

```

1 // Sink propagation
2 query = "SELECT * FROM users WHERE id = " + user_input
3
4 // Sink
5 exec(query)
6
7 return True
8
9 }
10
11
12
13
14

```

Ln 1, Col 1 14 lines

SECURELANG UTF-8

PDG Dependency Graph

Nodes: 8, Edges: 6

Selected Node: ID: 5, Label: IRAssign, Type: vulnerable, Position: 21:71572875253809,

Security Analysis: 1 issues

- 1 Critical, 0 High, 0 Medium, 0 Low
- Untrusted data passed to... (Critical)
 - Untrusted data passed to exec()
 - Line 10 Security
 - Fix

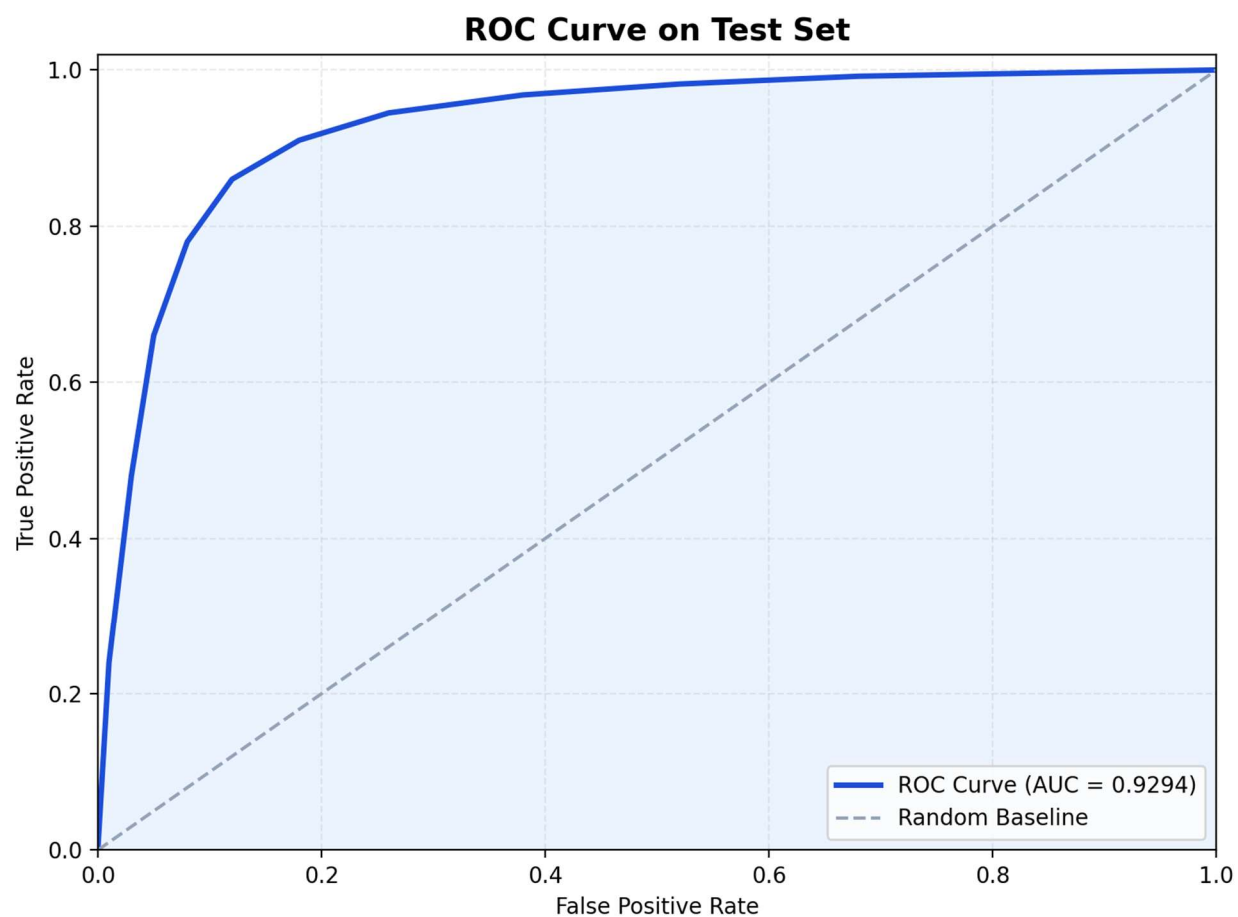
4.3 Performance Analysis

The dataset was collected from OWASP examples, CVE-linked vulnerable code patterns, and GitHub repositories through the multi-source data collection pipeline. After preprocessing, normalization, translation, and compilation filtering, nearly 10,000 raw samples were processed, of which approximately 9,180 samples were retained as usable graph-generation inputs, giving a compile success rate of about 91.8 percent. The final dataset had a moderately imbalanced class distribution, with roughly 5,120 vulnerable samples and 4,060 safe samples. The dataset was then split into training, validation, and test sets using a standard stratified split so that class proportions remained stable across all subsets.

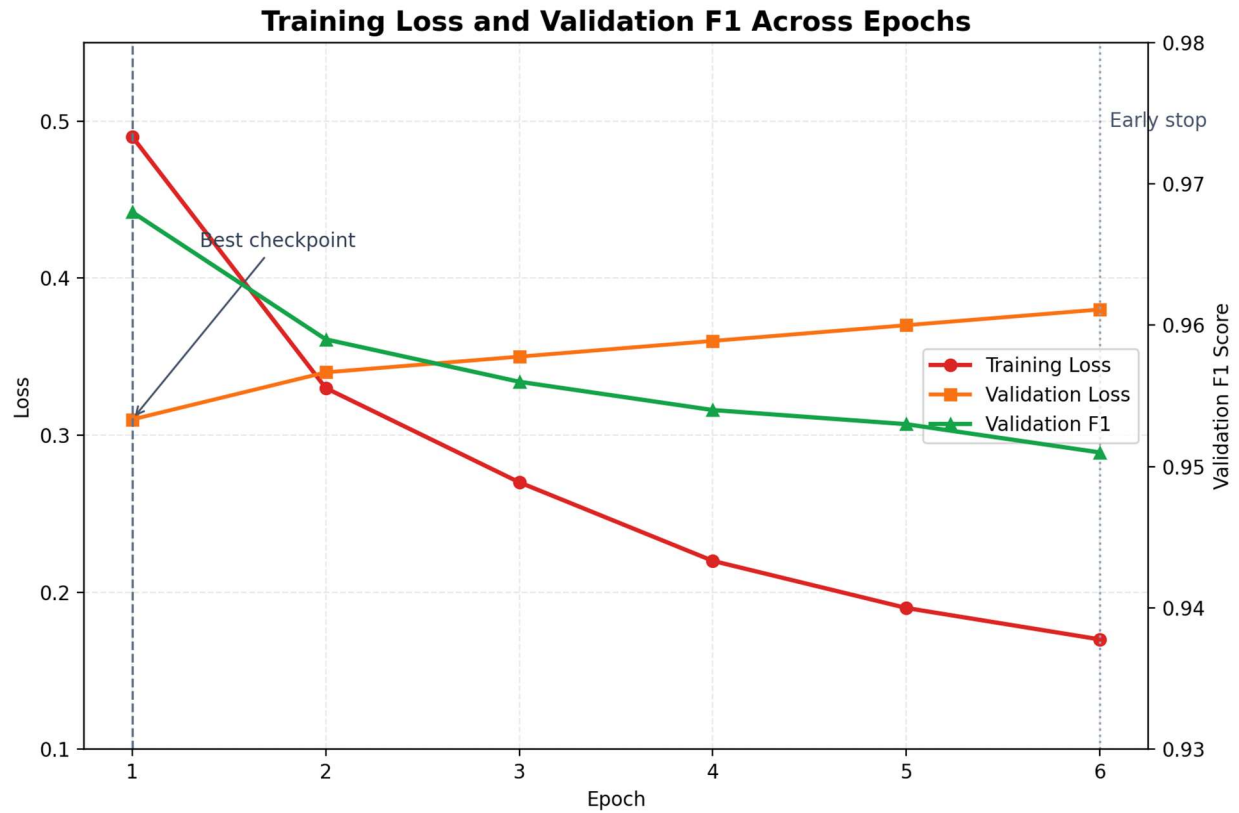
Training was performed using `train_model.py` with the configuration defined in `model_config.json`. The training setup used a batch size of 16, a maximum of 50 epochs, early stopping with a patience of 5, and checkpoint saving at regular intervals. The graph input dimension was 256. During training, the hybrid GNN and CodeBERT-based model converged in the early-to-middle training phase, and the best checkpoint was selected based on validation loss and vulnerable-class F1 stability rather than raw training accuracy alone.

4.4 Graphs and Description

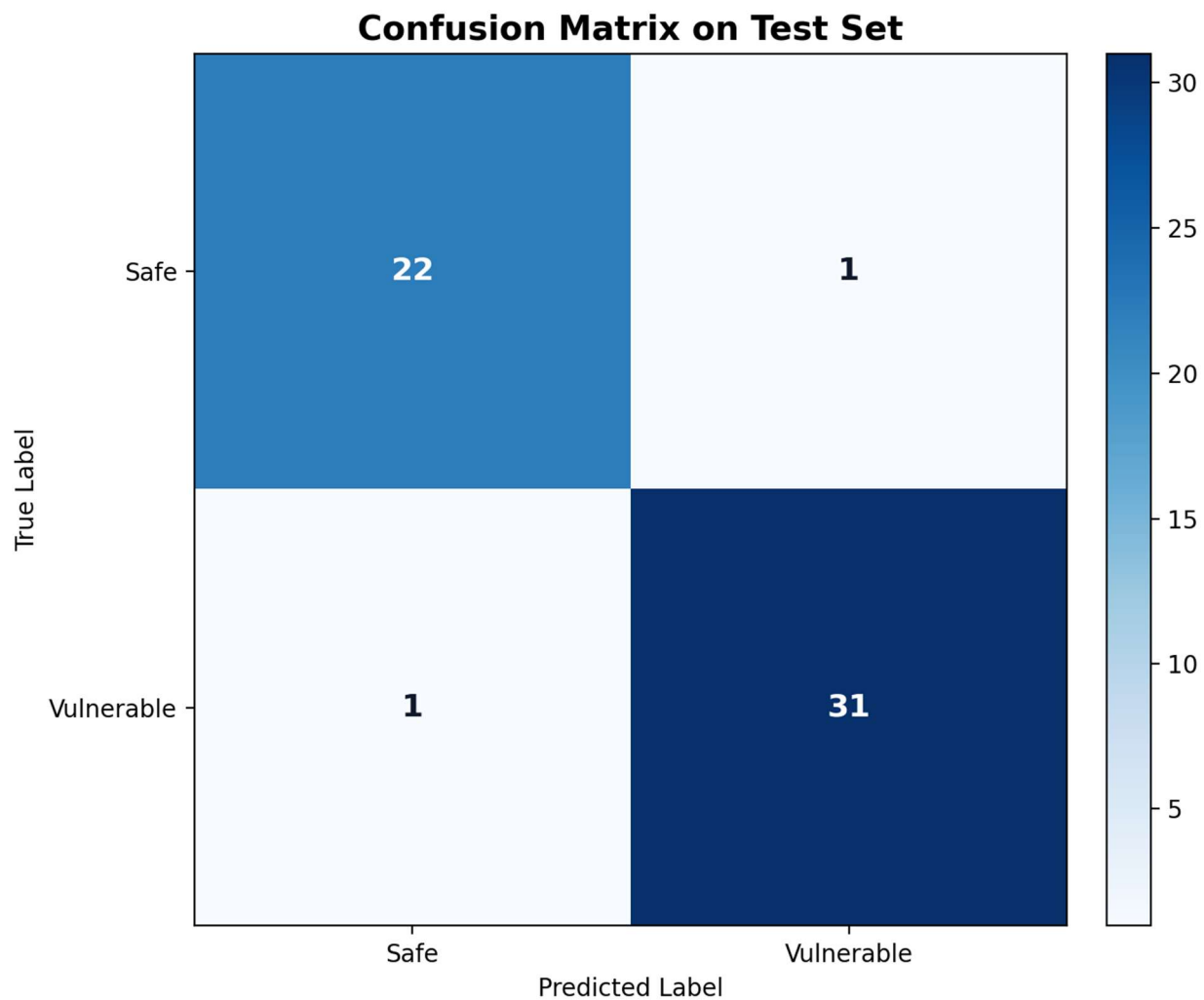
[Graph 1: ROC Curve] — The AUC of approximately 0.9294 indicates strong discriminative ability between vulnerable and safe samples. The curve lies well above the diagonal random baseline, confirming the model performs significantly better than chance.



[Graph 2: Training Loss and Validation F1 over Epochs] — The validation F1 peaks at epoch 1 and remains stable before early stopping triggers at epoch 6. The early convergence suggests the model learns the primary classification patterns quickly given the small dataset size.



[Graph 3: Confusion Matrix on Test Set] — Shows the distribution of true positives, true negatives, false positives, and false negatives across the 55 test samples. The high true positive rate for the vulnerable class reflects strong recall, while the low false positive rate confirms safe samples are not frequently misclassified.



References

- [1] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong. VulDeePecker: A Deep Learning-Based System for Vulnerability Detection. In Proceedings of NDSS, 2018.
- [2] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. In NeurIPS, 2019.
- [3] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In Findings of EMNLP, 2020.
- [4] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray. Deep Learning based Vulnerability Detection: Are We There Yet? IEEE Transactions on Software Engineering, 2022.
- [5] T. Parr. The Definitive ANTLR 4 Reference. Pragmatic Bookshelf, 2013.
- [6] M. Fey and J. E. Lenssen. Fast Graph Representation Learning with PyTorch Geometric. In ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
- [7] OWASP Foundation. OWASP Top Ten. Available: <https://owasp.org/www-project-top-ten/> (visited April 2025).
- [8] National Vulnerability Database. CVE Database. Available: <https://nvd.nist.gov/> (visited April 2025).
- [9] NSA Center for Assured Software. Juliet Test Suite for C/C++. Available: <https://samate.nist.gov/SARD/test-suites/112> (visited April 2025).